



EDITORA AGÊNTICA

MANUAL COMPLETO

CONTEXT USAGE — ENTENDENDO O CONTEXTO DE LLMs PARA INICIANTEs

NÍVEL BÁSICO

Heverton Eduardo Peres

ESPECIALISTA EM MARKETING E DESENVOLVIMENTO DE SOLUÇÕES

Heverton Eduardo Peres

MANUAL COMPLETO

**CONTEXT USAGE — ENTENDENDO O CONTEXTO DE LLMs PARA
INICiantes**

Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

Brasil
2026

Sumário

MANUAL COMPLETO — CONTEXT USAGE: Entendendo o Contexto de LLMs	5
1. Introdução	5
1.1 Para quem é este manual	5
1.2 O que você vai aprender	5
2. Conceitos Fundamentais	6
2.1 O que é um LLM	6
2.2 O que é um Token	6
2.3 O que é Context Window	7
2.4 O que é Context Usage	7
3. As Partes do Context Usage	8
3.1 System Prompt (Instruções de Sistema)	8
3.2 Conversation History (Histórico)	8
3.3 Tool Calls e Outputs (Ferramentas)	9
3.4 File Contents (Arquivos Lidos)	9
3.5 Metadata e Overhead	9
4. Como Funciona em Cada Harness	10
4.1 ChatGPT (OpenAI)	10
4.2 Claude Code (Anthropic)	11
4.3 OMP + Oh My Pi (OpenCode)	11
4.4 Cursor	12
4.5 Windsurf (Codeium)	12
4.6 Gemini CLI (Google)	13
4.7 Grok CLI (xAI)	13
4.8 Codex CLI (OpenAI)	14
4.9 Antigravity	14
4.10 Kiro (Amazon)	15
5. Tabela Comparativa: Contexto por Harness	16
5.1 O que muda na prática	16
6. O Ciclo de Vida do Contexto	17
6.1 Início da sessão	17
6.2 Durante a conversa	17
6.3 Quando o contexto fica cheio	17
7. Como Otimizar o Uso do Contexto	18
7.1 Para Usuários	18

7.2 Para Desenvolvedores	18
8. Glossário	19
9. Perguntas Frequentes	19

MANUAL COMPLETO — CONTEXT USAGE: Entendendo o Contexto de LLMs

1. Introdução

1.1 Para quem é este manual

Este manual é para **iniciantes** que querem entender como funciona o “contexto” de um modelo de linguagem (LLM). Se você já usou ChatGPT, Claude, Gemini ou qualquer assistente de IA e se perguntou “por que ela esqueceu o que eu disse lá atrás?” ou “por que a resposta ficou ruim depois de muito tempo conversando?”, este manual explica tudo.

1.2 O que você vai aprender

- O que é um LLM e como ele “pensa”
 - O que é o **context window** (janela de contexto)
 - O que é **CONTEXT USAGE** (consumo de contexto)
 - Quais partes compõem o consumo de contexto
 - Quem gerencia cada parte
 - Como funciona em cada harness (ChatGPT, Claude Code, OMP, Cursor etc.)
 - Como otimizar o uso do contexto
-

2. Conceitos Fundamentais

2.1 O que é um LLM

LLM significa **Large Language Model** (Modelo de Linguagem Grande). É um programa de inteligência artificial treinado para entender e gerar texto em linguagem humana.

Como funciona na prática: quando você digita uma mensagem para um assistente de IA, o LLM recebe seu texto e gera uma resposta “prevendo” o próximo **token** mais provável, repetidamente, até formar uma resposta completa.

Termos importantes:

Termo	Significado
LLM	Large Language Model — modelo de linguagem grande
Modelo	O “cérebro” treinado que gera texto (ex.: Claude, GPT, Gemini)
Token	A unidade básica que o modelo processa
Prompt	O texto de entrada que você envia para o modelo
Response	O texto que o modelo gera como resposta
Inference	O ato do modelo “pensar” e gerar uma resposta

2.2 O que é um Token

Token é a unidade básica de processamento de um LLM. Não é exatamente uma palavra — é um pedaço de texto que o modelo usa para entender e gerar linguagem.

Regra prática: em português e inglês, **1 token $\approx$ 3/4 de uma palavra**.

Texto	Tokens aproximados
“oi”	1 token
“A capital do Brasil é Brasília”	7 tokens
Um parágrafo de 100 palavras	~130 tokens
Um arquivo de código (200 linhas)	~2.000 tokens
Este manual inteiro	~8.000 tokens

Por que importa: todos os limites de um LLM são medidos em tokens — o tamanho máximo da conversa, o custo por uso, a velocidade de resposta.

2.3 O que é Context Window

Context Window é a **quantidade máxima de tokens** que um modelo pode processar de uma vez. É a “memória de curto prazo” do modelo.

Analogia: imagine uma mesa de trabalho. O context window é o tamanho da mesa. Tudo que está na mesa (papeis, anotações, documentos) o modelo pode ver e usar. Tudo que está fora da mesa, o modelo não enxerga.

Tamanhos típicos por modelo:

Modelo	Context Window	Equivalente
GPT-4o	128k tokens	~96 mil palavras
Claude 3.5 Sonnet	200k tokens	~150 mil palavras
Claude Opus 4	200k tokens	~150 mil palavras
Gemini 1.5 Pro	2M tokens	~1.5 milhão de palavras
DeepSeek V4	128k tokens	~96 mil palavras
Llama 3.1 70B	128k tokens	~96 mil palavras

2.4 O que é Context Usage

Context Usage é a **quantidade do context window que já está sendo usada** em um momento qualquer da conversa.

Formato típico: Context: 45,230 / 200,000 tokens (22.6%)

3. As Partes do Context Usage

O consumo total é formado por **5 componentes principais**:

CONSUMO TOTAL = System Prompt + History + Tool Outputs + File Contents + Metadata

3.1 System Prompt (Instruções de Sistema)

O **System Prompt** é o conjunto de **instruções invisíveis** que o modelo recebe ANTES de qualquer mensagem sua. É o “manual de operações” que diz ao modelo quem ele é, como se comportar, quais ferramentas tem e quais regras seguir.

O que contém:

Componente	Exemplo
Identidade	“You are a helpful coding assistant”
Regras	“Always respond in PT-BR”
Ferramentas	Read, Bash, Edit, Grep, etc.
Skills	“use fable-method before complex tasks”
Convenções	“Use pytest for tests”

Tamanho típico por harness:

Harness	System Prompt
ChatGPT (web)	~500 tokens
Claude Code (básico)	~5.000 tokens
OMP + Oh My Pi	~35.000 tokens
Cursor (com regras)	~10.000 tokens

Quem gerencia: o **harness** monta o system prompt combinando suas instruções + plugin + projeto + tools.

3.2 Conversation History (Histórico)

Todas as mensagens trocadas durante a sessão:


```
[User] → "Crie uma função em Python"
[Assistant] → "Aqui está a função..."
[User] → "Adicione tratamento de erro"
[Assistant] → "Adicionei o try/except..."
```

Tamanho: cada turno gasta ~500-2.000 tokens. Uma sessão longa com 50 turnos pode consumir ~50k tokens.

Quem gerencia: o harness mantém e decide quando compactar/truncar.

3.3 Tool Calls e Outputs (Ferramentas)

Quando o modelo usa uma tool (ler arquivo, rodar comando), o resultado entra no contexto:

```
Tool call: read("src/main.py")
Tool result: [conteúdo – 2.000 tokens]
```

Tamanho: um arquivo lido pode consumir 500-10.000 tokens. É frequentemente a **maior fonte** de consumo.

Quem gerencia: modelo decide O QUE ler; harness limita QUANTO retorna.

3.4 File Contents (Arquivos Lidos)

Sinônimo parcial de tool outputs, mas especificamente o conteúdo de arquivos carregados no contexto.

Exemplos de consumo:

Arquivo	Tokens
Python pequeno (50 linhas)	~500
Python médio (200 linhas)	~2.000
TypeScript grande (500 linhas)	~5.000
CLAUDE.md extenso	~3.500

3.5 Metadata e Overhead

Informações auxiliares: tags XML, timestamps, IDs de tools, formatação.

Componente	Tamanho
Tags XML por mensagem	~100 tokens
Mensagens de sistema	~50-200 tokens cada
Formatação JSON	~50 tokens por mensagem

Quem gerencia: exclusivamente o harness.

4. Como Funciona em Cada Harness

Esta é a parte mais importante. O conceito de contexto é universal, mas cada harness implementa de forma diferente.

4.1 ChatGPT (OpenAI)

ChatGPT – Contexto
System Prompt: ~500 tokens (instruções base da OpenAI)
Conversation: variável (histórico visível ao usuário)
Memory (avançado): persistente (lembra de sessões anteriores)
Tools: web browse, code (resultados entram no contexto)

Particularidades: - **System Prompt enxuto** (~500 tokens) — a OpenAI gerencia internamente - **Memory persistente** — lembra de conversas anteriores (funcionalidade opcional) - **Compactação automática** — quando o contexto enche, resume silenciosamente - **Não expõe** o consumo exato ao usuário (mostra apenas barra visual) - **Modelos:** GPT-4o (128k), GPT-4o-mini (128k), o1 (200k)

4.2 Claude Code (Anthropic)

Claude Code – Contexto	
System Prompt:	~5.000 tokens (role + tools + project rules)
Conversation:	variável (todas as mensagens visíveis)
Tool Outputs:	truncados (output > limite é cortado)
CLAUDE.md:	~3-10k tokens (regras do projeto carregadas)

Particularidades: - **CLAUDE.md carregado** automaticamente no system prompt - **Skills do superpowers** adicionadas ao system prompt - **Truncamento de tools** — outputs grandes são cortados - **Compactação inteligente** — resume conversas longas - **Exibe** contexto na barra de status do terminal - **Modelos:** Claude Opus 4 (200k), Claude Sonnet 4 (200k)

4.3 OMP + Oh My Pi (OpenCode)

OMP – Contexto	
System Prompt:	~35.000 tokens
└─ OMP base:	~5.000 tokens
└─ Skills (90+):	~15.000 tokens
└─ Tool schemas:	~10.000 tokens
└─ CLAUDE.md:	~3.000 tokens
└─ Runtime rules:	~2.000 tokens
Conversation:	variável (histórico completo)
Tool Outputs:	truncados (headroom comprime logs > 7 linhas)
Skills dinâmicas:	sob demanda (carregadas quando invocadas)

Particularidades: - **System Prompt ENORME** (~35k tokens) — maior de todos os harnesses - **90+ skills** listadas no system prompt (mesmo sem serem usadas) - **Skills dinâmicas** — quando invocadas, o conteúdo completo entra no contexto - **Token Economy** — caveman, headroom, lean-ctx reduzem consumo - **CLAUDE.md extenso** — regras da fábrica de livros - **Plugins** — oh-my-opencode-slim adiciona muita informação - **Modelos variados** — mimo-v2.5 (1M ctx), deepseek-v4-flash-free (200k ctx) - **Exibe** na barra inferior do terminal

4.4 Cursor

Cursor – Contexto	
System Prompt:	~10.000 tokens (IDE context + .cursorrules)
Codebase Index:	variável (embeddings do projeto inteiro)
Conversation:	variável (chat + inline edits)
Menções:	sob demanda (arquivos, pastas, docs específicos)

Particularidades: - **Codebase Index** — indexa todo o projeto em embeddings para busca semântica - **Menções** — você pode puxar arquivos específicos com o prefixo `file.py` - **.cursorrules** — regras do projeto no system prompt - **Contexto de IDE** — aware de aberturas de arquivo, erros, etc. - **Dual mode** — chat normal + inline edit (com contextos separados) - **Modelos:** GPT-4o, Claude Sonnet, Cursor Small (próprio)

4.5 Windsurf (Codeium)

Windsurf – Contexto	
System Prompt:	~8.000 tokens (IDE + flow awareness)
Cascade Context:	variável (aware de fluxo de trabalho)
Codebase Index:	embeddings

(busca semântica no projeto)
Supercomplete: predição inline (contexto de linha atual)

Particularidades: - **Cascade** — aware do fluxo de trabalho (entende que você está fazendo deploy, teste etc.) - **Supercomplete** — predição inline com contexto da linha atual - **Codebase indexing** — busca semântica no projeto inteiro - **.windsurfrules** — regras do projeto

4.6 Gemini CLI (Google)

Gemini CLI – Contexto
System Prompt: ~3.000 tokens (instruções + GEMINI.md)
Conversation: variável (histórico da sessão)
Context Window: 1M-2M tokens (MAIOR de todos os modelos)
GEMINI.md: ~2-5k tokens (regras do projeto)

Particularidades: - **Context Window gigante** (1M-2M tokens) — muito mais espaço que outros - **GEMINI.md** — equivalente ao CLAUDE.md do Claude Code - **Menos tools** disponíveis que OMP/Claude Code - **OAuth** — autenticação via conta Google (gratuita) - **Compactação** menos necessária por ter tanto espaço

4.7 Grok CLI (xAI)

Grok CLI – Contexto
System Prompt: ~5.000 tokens (instruções + config.toml)
Conversation: variável (histórico da sessão)

Fork Model:	contexto separado (pensamento paralelo)
Plugins:	sob demanda (vercel, etc.)

Particularidades: - **Fork model** — pode usar um modelo secundário para “pensar” em paralelo - **config.toml** — configuração via TOML - **Plugins** — extensíveis via marketplace - **Modelo:** Grok 4.5 (131k tokens)

4.8 Codex CLI (OpenAI)

Codex CLI – Contexto
System Prompt: ~5.000 tokens (instruções + config.toml)
Conversation: variável (histórico da sessão)
Approvals: interação manual (usuário aprova cada ação)
config.toml: ~1k tokens (MCP servers + settings)

Particularidades: - **Aprovação manual** — cada tool call precisa de OK do usuário - **Modelos:** GPT-5, GPT-5-mini, GPT-5-nano - **MCP servers** — suporta servers externos - **config.toml** — configuração via TOML

4.9 AntigraVity

Antigravity – Contexto
System Prompt: ~5.000 tokens (baseado em VS Code)
Extensions: sob demanda

(extensões VS Code)

Copilot-style: inline completion (predição de código)
--

4.10 Kiro (Amazon)

Kiro – Contexto

System Prompt: ~5.000 tokens (instruções + skills)

Specs: estruturado (requisitos formais)
--

Steering Files: ~2-5k tokens (regras do projeto)

5. Tabela Comparativa: Contexto por Harness

Harness	System Prompt	Context Window	Compactação	Exibe Uso?	Skills
ChatGPT	~500 tok	128-200k	Automática silenciosa	Barra visual	Não
Claude Code	~5k tok	200k	Inteligente	Barra de status	Sim (super-powers)
OMP	~35k tok	128k-1M	Token Economy	Barra inferior	Sim (90+)
Cursor	~10k tok	128k	Automática	Painel de tokens	Não
Windsurf	~8k tok	128k	Automática	Indicador	Não
Gemini CLI	~3k tok	1-2M	Mínima (muito espaço)	Básico	Sim
Grok CLI	~5k tok	131k	Automática	Básico	Sim (market-place)
Codex CLI	~5k tok	128k	Manual (aprovação)	Básico	Sim (MCP)
Antigravity	~5k tok	128k	Automática	Indicador	Não
Kiro	~5k tok	128k	Automática	Básico	Sim

5.1 O que muda na prática

ChatGPT: você não precisa se preocupar — a OpenAI gerencia tudo. Mas tem menos controle.

Claude Code: system prompt enxuto, bom para projetos médios. CLAUDE.md carregado automaticamente.

OMP: system prompt enorme (~35k tokens), mas com mais ferramentas e skills. Para projetos grandes, considere ativar lean-ctx/caveman.

Cursor: bom equilíbrio entre contexto de IDE e LLM. Codebase index ajuda em projetos grandes.

Gemini CLI: espaço GIGANTE (1-2M tokens). Raramente precisa de compactação. Bom para revisar arquivos inteiros.

6. O Ciclo de Vida do Contexto

6.1 Início da sessão

[System Prompt] ← ~35k tokens (OMP) ou ~500 tokens (ChatGPT)
[Conversation] ← 0 tokens
[Tool Outputs] ← 0 tokens

6.2 Durante a conversa

A cada interação, o consumo cresce:

Turno 1: +5k tokens → Total: 40k (20%)
Turno 5: +15k tokens → Total: 55k (27.5%)
Turno 10: +25k tokens → Total: 70k (35%)
Turno 20: +50k tokens → Total: 95k (47.5%)
Turno 30: +80k tokens → Total: 125k (62.5%)

6.3 Quando o contexto fica cheio

Faixa	Status	O que acontece
0-20%	 Livre	Funcionamento ideal
20-50%	 Normal	Funcionamento bom
50-70%	 Atenção	Começa a compactar
70-85%	 Crítico	Compactação agressiva
85-100%	 Perigo	Erros, instabilidade

7. Como Otimizar o Uso do Contexto

7.1 Para Usuários

Ação	Economia
<code>read("file.py:50-100")</code> em vez de arquivo inteiro	~80%
<code>grep</code> antes de <code>read</code>	~60%
Ser específico nos pedidos	~40%
Modo caveman (OMP)	~65%
headroom em logs	~50%
lean-ctx (grep primeiro)	~40%

7.2 Para Desenvolvedores

Ação	Impacto
CLAUDE.md enxuto	Reduz system prompt
Skills específicas	Evita carregar desnecessárias
Tools limitadas	Reduz schemas
.gitignore bem configurado	Evita ler arquivos desnecessários

8. Glossário

Termo	Definição
LLM	Large Language Model
Token	Unidade básica (~3/4 palavra)
Context Window	Limite máximo de tokens
Context Usage	Consumo atual do contexto
System Prompt	Instruções invisíveis ao modelo
Conversation History	Mensagens trocadas
Tool Call	Chamada de ferramenta
Tool Output	Resultado da ferramenta
Harness	Programa que gerencia o LLM
Plugin	Extensão do harness
Skill	Conhecimento especializado
Truncation	Corte de conteúdo antigo
Compaction	Resumo de conteúdo antigo
Token Economy	Estratégias de redução
Inference	Geração de resposta pelo modelo

9. Perguntas Frequentes

“Por que meu assistente esqueceu o que eu disse?” → O contexto ficou cheio e o harness compactou as mensagens antigas.

“Por que a resposta ficou ruim depois de muito conversar?” → O contexto está consumindo 80%+ do limite. Menos “espaço” para pensar.

“Posso aumentar o context window?” → Não diretamente. Use um modelo com contexto maior ou otimize o consumo.

“O que acontece quando enche completamente?” → Truncamento automático, resumo, ou nova sessão.

“É o mesmo em todos os harness?” → O conceito é universal, mas cada harness implementa diferente (ver seção 4).

Manual mantido por `scripts/atualizar-documentacao.py` — não edite o PDF à mão; edite este `.md` e recompile.

